Where Does Your Data Live?

Week 1 — Pointers, Dynamic Memory, and Simple Contracts

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 1

Today's Three Questions

1. Where does a running C program keep its data?
2. How can we use memory that we request while the program runs?
3. How can we describe a data structure before implementing it?

Plan

First draw the memory. Then use one pointer. Then write one simple contract.

A Variable Has a Place

Example

`int marks = 80;` stores the value 80 somewhere in memory.

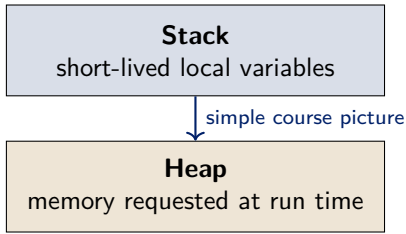
- The variable has a name: `marks`.
- It has a value: 80.
- It also has an address: its place in memory.

An address is like a house number. It tells us where to look.

Section 1

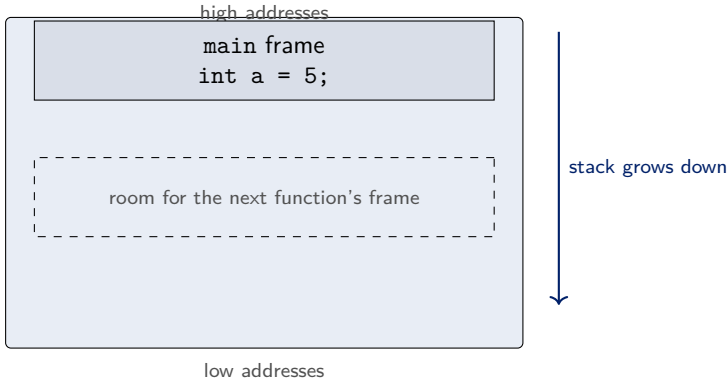
Memory

Two Places to Remember



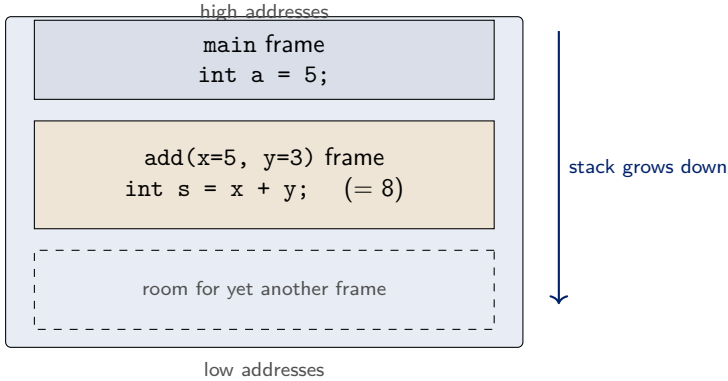
- Local variables usually live in the stack.
- Dynamic memory lives in the heap.

How the Stack Grows — Step 1: `main` starts



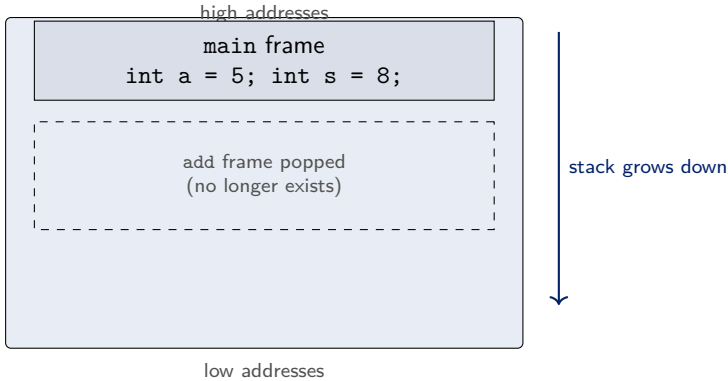
- Starting `main` creates one frame holding its local variables.
- Every new function call pushes a new frame just below the current one.

How the Stack Grows — Step 2: `add` is called



- The call to `add` pushed a second frame with its parameters and locals.
- The new frame sits at a **lower** address than `main`'s frame.

How the Stack Grows — Step 3: `add` returns



- Returning popped the frame, and its memory is available again.
- The answer 8 was copied back into `main`'s frame.

The Stack: Automatic Storage

Local variable

A variable declared inside a function is created when the function starts.

- It is available while that function is running.
- It disappears when the function returns.
- The program manages this basic lifetime automatically.

Example

`int score = 80;` inside a function is a local variable.

Why Do We Need the Heap?

A changing problem

The user enters the number of records only after the program starts.

- We may need space for 3 records today and 300 tomorrow.
- We may want the records to remain after a helper function returns.
- The heap gives us memory whose size and lifetime we can request at run time.

The heap is useful when the data must be flexible.

Check Your Prediction

Which variable can outlive the function that created it?

```
int score = 80;          or          int *p = malloc(sizeof(int));
```

- The local variable `score` belongs to its function call.
- The heap block remains until the program releases it with `free`.
- The pointer is only the handle; the block is the data storage.

Turn and talk

What must remain available if another function needs the record later?

The exact call for this — `malloc` and `sizeof` — comes in a few slides.

Section 2

Pointers

What Is a Pointer?

Definition

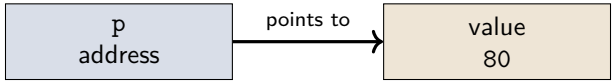
A pointer is a variable that stores an address [3].

- An ordinary variable stores a value such as 80.
- A pointer stores a place such as “where the score lives”.
- The type tells C what kind of value is at that place.

One declaration

`int *p;` means: `p` can point to an `int`.

A Pointer Picture



- p does not contain the score itself.
- p tells us where the score can be found.

A Pointer Can Reach a Node

A two-field node

```
struct node {
  int data;
  struct node *next;
};
```

- `data` stores the value.
- `next` stores the address of another node.
- If there is no next node, set `next = NULL`.

This is the basic building block for a linked list in a later week.

The Safe Empty Pointer

`NULL`

`NULL` means that a pointer is not pointing to a valid object.

- Use it to show that a pointer is currently empty.
- Check a pointer before using the object it should point to.
- Never try to read through an empty pointer.

Picture

`struct node *head = NULL;` means "the list is empty".

Section 3

One Heap Block

Requesting One Integer

The basic pattern

```
int *p = malloc(sizeof(int));
if (p == NULL) return 1;
*p = 80;
```

- `malloc` requests bytes from the heap.
- `sizeof(int)` asks C for the right number of bytes.
- `p` stores the address of the requested space.

Allocate a Complete Node

A safer engineering pattern

```
struct node *p = malloc(sizeof(struct node));
if (p == NULL) return 1;
p->data = 42;
p->next = NULL;
```

- `sizeof(struct node)` asks for exactly one node's bytes.
- `p->data` means: the `data` field of the node `p` points at.
- Initialising every field makes the node's state predictable.
- This node has a clear end: `p->next == NULL`.

Audit the Node Before You Ship It

Review checklist

A small program is not finished when it compiles.

1. Was the allocation checked against `NULL`?
2. Was every field initialised before the node was used?
3. Is there exactly one clear owner responsible for `free(p)`?
4. Can the program reach the node later, or has its pointer been lost?

Engineering habit

Test the normal path and the allocation-failure path.

Trace It Before Running It

After these lines, what is true?

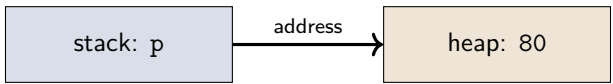
```
int *p = malloc(sizeof(int)); *p = 80;
```

1. `p` contains an address.
2. The heap block at that address contains 80.
3. The name `p` itself is not the heap block.

Challenge

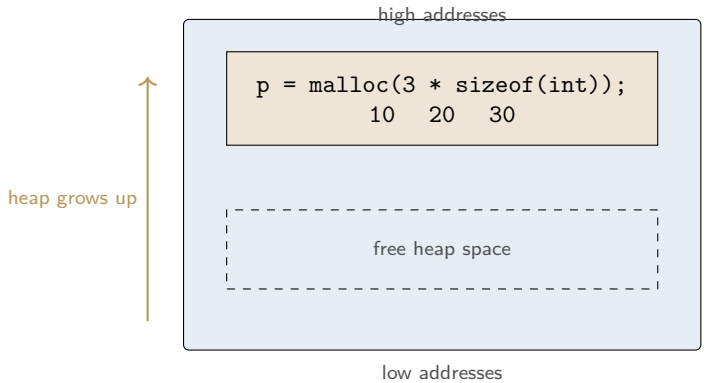
Which line would fail if `p == NULL`?

What Happened?



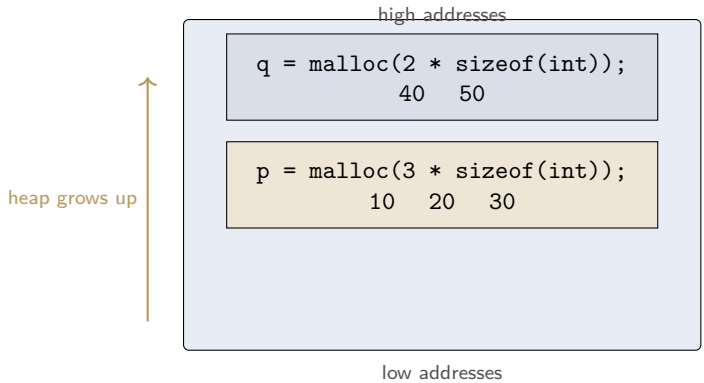
1. C found a free heap block.
2. The address was stored in `p`.
3. We used `*p` to write the value 80 there.

How the Heap Grows — Step 1: a block of three ints



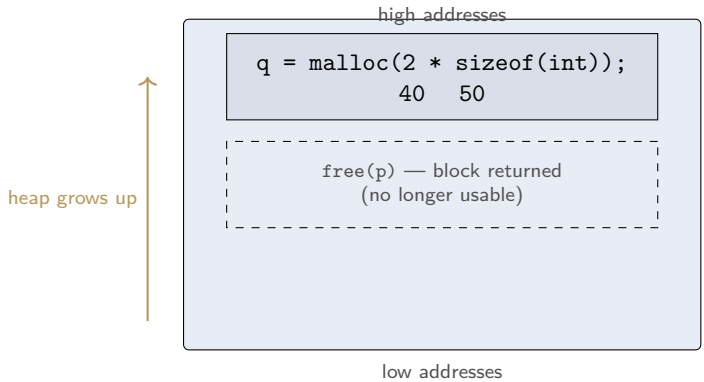
- One `malloc` with room for three integers **grew the used heap** upward.
- The block survives after the allocating function returns.

How the Heap Grows — Step 2: a second block



- The second request placed q's block at a **higher** address.
- Blocks are independent; each keeps its own data.

How the Heap Grows — Step 3: `free` returns a block



- `free(p)` gives `p`'s block back to the system.
- `q`'s block is untouched, and the used heap can shrink again.

Returning the Memory

When the block is no longer needed

```
free(p);
p = NULL;
```

- `free(p)` gives the heap block back to the system.
- Setting `p` to `NULL` makes the old pointer visibly empty.
- A block requested with `malloc` should eventually be freed.

Request memory, use it, return it.

Two Beginner Mistakes

- Memory leak: forget to call `free`; the block stays reserved.
- Dangling pointer: use a pointer after its block was freed.

Simple habit

Keep track of who requested the memory and who will return it.

Section 4

Simple Contracts

What Is an ADT?

Abstract Data Type

An ADT describes what we can do with data, without saying how it is stored [2, 1].

- It is a promise about operations.
- The implementation is the code and memory arrangement behind the promise.
- Different implementations can follow the same promise.

An ADT has two parts: a declaration of data, and a declaration of operations [1].

This Course Is a Tour of ADTs

The pattern you will see every week

Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Disjoint Sets, Hash Tables, and Graphs are all commonly used ADTs [1].

- Each one starts the same way: state the contract, *then* implement it.
- Weeks 3–12 work through this list one ADT at a time.
- Today's "bag" is a warm-up for that pattern, not a one-off example.

Watch for this shape: "What is a Stack?" then "Stack ADT" then "Implementation" — every week, same order.

Example: A Very Small Bag

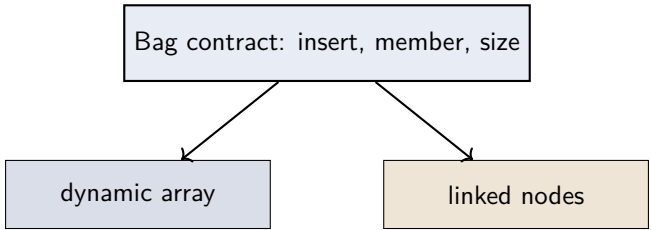
The contract

A bag of integers supports three operations:

1. `insert(x)` — put `x` into the bag.
2. `member(x)` — answer whether `x` is present.
3. `size()` — report how many items are present.

The contract does not yet say whether the bag uses an array or a list.

One Contract, Two Possible Implementations



The key distinction

Same operations, different internal arrangement.

Contract or Implementation?

Classify each statement

Is it part of the contract, or part of one implementation?

1. "`member(7)` answers yes or no."
2. "The values are stored in an array."
3. "`size()` reports the number of values."

The first and third describe what users can ask. The second describes how.

Week 1 Summary

- The stack holds ordinary short-lived local variables.
- The heap holds memory requested while the program runs.
- A pointer stores an address; `NULL` means "points to nothing".
- Use `malloc`, check the result, use the block, and call `free`.
- An ADT describes operations; a data structure implements them.
- The rest of this course is a tour of ADTs: contract first, implementation second.

Next week

We will learn how to talk about the amount of work an operation needs.

References I

Narasimha Karumanchi.

Data Structures And Algorithms Made Easy.

CareerMonk Publications, 2017.

This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4th edition, 2011.

Niklaus Wirth.

Algorithms and Data Structures.

Author (freely distributed), 2004.

Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.